

Universidad del Istmo de Guatemala

Tecnología IP · Catedrático: Jorge Lara

Manual técnico de instalación y operación

Clúster de alta disponibilidad PostgreSQL 16 + Patroni + etcd + HAProxy
en tres equipos macOS / Linux

MANUAL TÉCNICO

Didvin Nohel Estrada Pineda
Carné 14092

Guatemala, octubre de 2026

Manual técnico de instalación y operación

Autor	Didvin Nohel Estrada Pineda
Carné	14092
Curso	Tecnología IP
Catedrático	Jorge Lara
Repositorio	github.com/NohelEstradaP/ha_db_ip
Documento complementario	`INFORME.md` (diseño, fundamentos y resultados)

Índice

1. Acerca de este manual
2. Descripción del sistema
3. Requisitos
4. Estructura del repositorio
5. Instalación de un nodo
6. Puesta en marcha del clúster
7. Operación diaria
8. Panel web
9. Cambio de primario
10. Procedimientos ante fallas
11. Reinstalar o reemplazar un equipo
12. Pruebas de alta disponibilidad
13. Laboratorio local
14. Mantenimiento
15. Referencia de comandos
16. Solución de problemas
17. Glosario

1. Acerca de este manual

Este manual explica cómo instalar, poner en marcha, operar, probar y recuperar el clúster de alta disponibilidad descrito en el informe técnico. Está dirigido a quien administre los tres equipos.

Convenciones:

- Los comandos se ejecutan en la carpeta del proyecto (`~/hadb` o donde se haya clonado), salvo que se indique otra cosa.
- `n1`, `n2` y `n3` son los nombres de los nodos. `N1_IP`, `N2_IP` y `N3_IP` son sus IP (Tailscale `100.x.y.z` o LAN).
- `$` indica un comando de terminal. En los bloques de código se omite.
- ⚠ marca una acción que puede interrumpir el servicio o borrar datos.

Configuración de referencia (la del proyecto desde el 3 de noviembre de 2026; antes, n2 era una PC con Pop!_OS en 100.88.183.25):

Nodo	Equipo	Sistema	Contenedores	IP Tailscale
n1	MacBook Air M3	macOS · ARM64	Docker Desktop	100.73.72.12
n2	Dell Latitude 5420	Ubuntu 24.04 · x86-64	Docker Engine	100.69.167.56
n3	MacBook Pro	macOS · ARM64	OrbStack	100.114.31.68

2. Descripción del sistema

2.1 Qué hace

- Mantiene una base de datos PostgreSQL 16 (`hadb`) replicada en tres equipos.
- **Un** nodo es el **primario**: acepta lecturas y escrituras. Los otros dos son **réplicas** de solo lectura:
- la **réplica síncrona** debe confirmar cada `COMMIT` antes de que el primario responda al cliente;
- la **réplica asíncrona** recibe los cambios milisegundos después.
- Si el primario cae, la réplica síncrona lo reemplaza automáticamente.
- El nodo que regresa se re-sincroniza solo.

2.2 Servicios de cada nodo

Servicio	Contenedor	Función
etcd	`etcd`	Consenso entre los 3 nodos: quién es el líder. Necesita mayoría (2 de 3)
Patroni + PostgreSQL	`patroni`	Base de datos y su agente de alta disponibilidad
HAProxy	`haproxy`	Punto de entrada: 5000 siempre llega al primario y 5001 a las réplicas

Fuera de Docker, y de forma opcional, corren el **generador de carga** (`app/generador.py`) y el **panel web** (`app/panel.py`).

2.3 Puertos

Puerto	Protocolo	Servicio	Quién lo usa
2379	TCP	etcd (clientes)	Patroni de los 3 nodos, scripts
2380	TCP	etcd (pares)	etcd de los 3 nodos
5432	TCP	PostgreSQL	Réplicas, aplicación, `pg_rewind`
8008	TCP/HTTP	API REST de Patroni	HAProxy, panel, Patroni de los otros nodos
5000	TCP	HAProxy → primario (lectura/escritura)	Clientes sin soporte de varios hosts
5001	TCP	HAProxy → réplicas (solo lectura)	Consultas de lectura
7000	HTTP	Estadísticas de HAProxy	Navegador, panel
8080	HTTP	Panel web	Navegador
41641	UDP	Tailscale	Red en malla

2.4 Roles y estados que muestra Patroni

Rol	Significado
`Leader`	Primario: acepta escrituras
`Sync Standby`	Réplica síncrona: cada `COMMIT` espera su confirmación. Es la única que puede ser promovida
`Replica`	Réplica asíncrona

Estado	Significado
`running`	PostgreSQL en marcha. Es lo normal en el primario; en una réplica, todavía no recibe WAL
`streaming`	Réplica recibiendo cambios en tiempo real (normal)

Estado	Significado
`creating replica`	Copiando la base completa desde el primario
`starting` / `stopped`	Arrancando / detenido
`start failed`	No pudo arrancar: ver §10.3

3. Requisitos

3.1 Hardware (por equipo)

- 2 núcleos, **4 GB de RAM libres para Docker** (2 GB alcanzan para la carga de prueba) y 10 GB de disco libre.
- Cargador conectado durante las pruebas y la demostración.

3.2 Software

Sistema	Requisito	Lo instala `instalar.sh`
macOS 13 o posterior (Apple Silicon o Intel)	Homebrew, Docker Desktop **u** OrbStack, Tailscale, git, jq, Python ≥ 3.10	Sí (usa OrbStack si ya está)
Pop!_OS 22.04/24.04 o Ubuntu	Docker Engine + Compose v2, Tailscale, git, jq, Python 3 + venv, netcat	Sí

3.3 Red

- **Recomendado:** una cuenta de **Tailscale** con los 3 equipos en el mismo *tailnet*. Funciona en la misma red y en redes distintas.
- **Alternativa:** los 3 equipos en la misma LAN con IP fijas (`./instalar.sh --lan`).
- Si la red bloquea UDP, Tailscale usa relays (DERP): funciona igual, con más latencia.

3.4 Cuentas y secretos

- Cuenta de Tailscale, con **key expiry desactivado** para los 3 equipos.
- **4 contraseñas compartidas** por los 3 nodos, generadas por el instalador del primer equipo. **Guárdelas en un lugar seguro**: se necesitan para reinstalar un equipo.

Variable	Uso
`PG_SUPERUSER_PASSWORD`	Usuario `postgres`
`PG_REPLICATION_PASSWORD`	Usuario `replicator` (streaming)
`PG_REWIND_PASSWORD`	Usuario `rewind_user` (`pg_rewind`)
`APP_PASSWORD`	Usuario `app` (generador, panel, verificador)

4. Estructura del repositorio

```
ha_db_ip/
├── instalar.sh                 Prepara un equipo como nodo
├── docker-compose.yml          etcd + patroni + haproxy (idéntico en los 3 nodos)
├── .env.example                Plantilla del .env (el .env real no se versiona)
├── patroni/
│   ├── Dockerfile              postgres:16-bookworm + Patroni 4.0.4
│   └── patroni.yml             Configuración de Patroni y PostgreSQL
├── haproxy/haproxy.cfg         5000 → primario, 5001 → réplicas, 7000 → estadísticas
├── sql/esquema.sql             Tablas cuentas y movimientos
├── app/
│   ├── generador.py            Carga: transferencias idempotentes
│   ├── verificar.py            Integridad: compara los 3 nodos
│   ├── monitor.py              Estado en la terminal
│   ├── panel.py, panel.html    Panel web en tiempo real
│   └── requirements.txt        psycopg 3
├── scripts/                    Operación (ver §15.1)
├── laboratorio/                Los 3 nodos en una sola máquina
├── evidencias/                 Logs y CSV generados (no se versiona)
└── docs/img/                   Imágenes de la documentación
```

5. Instalación de un nodo

5.1 Antes de empezar

1. Cree la cuenta de Tailscale (https://login.tailscale.com) y defina qué equipo será `n1`, `n2` y `n3`.
2. **macOS:** desactive el **Receptor AirPlay**, que ocupa los puertos 5000 y 7000: *Ajustes del Sistema → General → AirDrop y Handoff → Receptor AirPlay*.
3. Si en el equipo hay un PostgreSQL local usando el 5432, deténgalo: - macOS: `brew services stop postgresql@16` - Linux: `sudo systemctl disable --now postgresql`

5.2 Ejecutar el instalador

```
git clone https://github.com/NohelEstradaP/ha_db_ip.git ~/hadb
cd ~/hadb
./instalar.sh
```

Opción	Efecto
(ninguna)	Instalación completa con Tailscale
`--lan`	Sin Tailscale: usa las IP de la red local
`--solo-env`	Solo vuelve a generar el `.env`
`--manual`	Ignora `cluster.env.enc` (si existe) y pide IP y contraseñas a mano
`-y`	Responde "sí" a las confirmaciones (excepto la de generar contraseñas)

Pasos que ejecuta el instalador:

Paso	macOS	Pop!_OS
Paquetes	Homebrew; git, jq, watch, Python si falta	`apt`: git, curl, jq, netcat, openssl, python3-venv
Docker	Usa OrbStack si está; si no, instala Docker Desktop y espera a que arranque	Repositorio oficial de Docker, `systemctl enable docker`, usuario al grupo `docker`
Tailscale	Instala la app y espera el inicio de sesión	`install.sh` oficial y `tailscale up` (muestra una URL para iniciar sesión)
Energía	`pmset -c sleep 0`	Enmascara `sleep/suspend/hibernate.target` e ignora el cierre de tapa
Hora	NTP	`timedatectl set-ntp true`
Puertos	Verifica 2379, 2380, 5432, 5000, 5001, 7000 y 8008; detecta AirPlay	Igual
`.env`	Pregunta nodo, IP y contraseñas (§5.3)	Igual
Firewall	—	Si `ufw` está activo, permite las IP de los 3 nodos
Python	`.venv` con psycopg (timeout 120 s, 10 reintentos)	Igual
Imágenes	`docker compose pull + build patroni`	Igual
Final	Ofrece ejecutar `./scripts/iniciar.sh`	Igual

5.3 Preguntas del instalador

Pregunta	Primer equipo	Segundo y tercer equipo
¿Qué nodo es este equipo?	`n1`, `n2` o `n3`	el que le corresponda
IP de n1 / n2 / n3	La propia aparece por defecto; las otras salen de `tailscale ip -4` en cada equipo o del panel de Tailscale	**Las mismas** que en el primero
¿Es el PRIMER equipo?	**`s`**: genera 4 contraseñas de 20 caracteres y las muestra	**`n`**: pegar las 4 contraseñas del primero
¿Levantar ahora este nodo?	`S` cuando los otros dos ya estén instalados	`S`

> ⚠ Responder `s` a "¿Es el PRIMER equipo?" en el segundo o tercer equipo genera contraseñas distintas, y ese nodo no podrá replicar.

5.4 Archivo `.env`

Lo genera el instalador. **No se versiona** (está en `.gitignore`).

```
NODE_NAME=n1                  # identidad de ESTE nodo
NODE_IP=100.73.72.12          # IP de ESTE nodo
N1_IP=100.73.72.12            # IP de los 3 nodos: idénticas en los 3 equipos
N2_IP=100.69.167.56
N3_IP=100.114.31.68
PG_SUPERUSER_PASSWORD=…       # idénticas en los 3 equipos
PG_REPLICATION_PASSWORD=…
PG_REWIND_PASSWORD=…
APP_PASSWORD=…
ETCD_ESTADO_INICIAL=new       # lo administra scripts/iniciar.sh; no editar
```

Para comprobar que los 3 equipos coinciden:

```
grep _IP .env       # debe ser idéntico en los 3 (salvo NODE_IP)
```

6. Puesta en marcha del clúster

6.1 Primer arranque

1. Con los 3 equipos instalados, ejecute **en los 3**, con pocos minutos de diferencia: `bash ./scripts/iniciar.sh` El script: - levanta los contenedores; - espera el quórum de etcd (necesita al menos 2 nodos); - espera a que haya un primario y a que **este** nodo quede sincronizado; - muestra el estado del clúster.
2. Desde cualquier equipo, compruebe la conectividad: `bash ./scripts/probar_puertos.sh # las 18 líneas deben decir OK`
3. **Una sola vez**, en cualquier equipo, cree la base de datos: `bash ./scripts/crear_bd.sh` Crea el usuario `app`, la base `hadb` y el esquema con 100 cuentas. Si ya existían, no hace nada. Resultado esperado: `cuentas: 100, suma: 1000000.00`.

6.2 Estado esperado

```
$ ./scripts/estado.sh
| Member | Host          | Role         | State     | TL | Lag in MB |
| n1     | 100.73.72.12  | Sync Standby | streaming |  1 |         0 |
| n2     | 100.69.167.56 | Leader       | running   |  1 |           |
| n3     | 100.114.31.68 | Replica      | streaming |  1 |         0 |
```

Debe haber exactamente **1 Leader**, **1 Sync Standby** y **1 Replica**, con lag 0. Justo después del arranque `Sync Standby` puede tardar unos 5 s en aparecer.

7. Operación diaria

7.1 Consultar el estado

Necesidad	Comando
Roles y lag	`./scripts/estado.sh` o `./scripts/pctl.sh list`
Historial de cambios de primario	`./scripts/pctl.sh history`
Contenedores de este nodo	`docker compose ps`
Logs de Patroni	`docker compose logs -f --tail 100 patroni`
Vista completa en el navegador	Panel web (§8)

7.2 Generar carga

```
./scripts/carga.sh iniciar          # generador de ESTE equipo en segundo plano, 1 tx/s
./scripts/carga.sh iniciar --tps 2  # más carga
./scripts/carga.sh estado
./scripts/carga.sh detener
```

- Cada transacción es una transferencia: 1 `INSERT` y 2 `UPDATE`, es decir, 3 cambios. A 1 tx/s son 180 cambios por minuto.
- **Recomendación:** un generador en **cada** equipo. Si solo corre en uno y ese equipo se apaga, la carga cae a 0.
- **Logs:** `evidencias/generador_<nodo>.log`. Detalle por transacción: `evidencias/generador_<nombre>-<fecha>.csv`.
- Para ver la carga en primer plano: `source scripts/env_app.sh && python app/generador.py --nombre n1`.

7.3 Verificar la integridad

```
source scripts/env_app.sh
python app/verificar.py             # con carga: revisa suma y faltantes en los 3 nodos
python app/verificar.py --esperar   # sin carga: además exige datos idénticos (md5)
```

Comprobación	Correcto
Suma de saldos en cada nodo	1,000,000.00
`faltan` (secuencias que faltan)	0
md5 de `cuentas` y `movimientos`	Igual en los 3 (con `--esperar`)
Primarios	Exactamente 1

Con la carga activa, los md5 pueden diferir por milisegundos de retraso. Detenga los generadores antes de usar `--esperar`.

7.4 Monitor de terminal

```
source scripts/env_app.sh && python app/monitor.py
```

Muestra el rol, el estado, el timeline y el lag de cada nodo, junto con las transacciones y cambios por minuto y la suma. Guarda un CSV en `evidencias/monitor_<fecha>.csv`.

7.5 Conectarse a la base de datos

Para	Cadena / comando
psql al primario desde un nodo	`docker exec -it patroni psql "host=$NODE_IP port=5000 dbname=hadb user=app"`
psql local (superusuario)	`docker exec -it patroni psql -U postgres -d hadb`
Aplicación con failover transparente	`host=N1_IP,N2_IP,N3_IP port=5432 dbname=hadb user=app target_session_attrs=read-write`
Herramienta gráfica (DBeaver, pgAdmin)	Host: IP de cualquier nodo · Puerto **5000** · Base `hadb` · Usuario `app`

7.6 Detener y arrancar

Acción	Comando	Efecto
Detener este nodo sin perder datos	`docker compose stop`	Si era primario, hay failover (~3 s)
Volver a arrancarlo	`./scripts/iniciar.sh`	Se reincorpora como réplica
Apagar **todo** el clúster	`docker compose stop` primero en las 2 réplicas y **al final en el primario**	—
Encender todo el clúster	`./scripts/iniciar.sh` **primero en el que era primario** y luego en los otros	El clúster vuelve con los mismos datos

> ⚠ `docker compose down -v` **borra** los datos del nodo (§11.3).

8. Panel web

8.1 Iniciar

```
./scripts/panel.sh                    # primer plano · http://localhost:8080
./scripts/panel.sh --fondo            # segundo plano (log en evidencias/panel.log)
./scripts/panel.sh --detener          # detiene el de segundo plano
./scripts/panel.sh --solo-lectura     # sin el botón "Hacer primario"
./scripts/panel.sh --puerto 8090      # otro puerto
```

Desde otro equipo o un celular conectado a Tailscale: `http://<IP del equipo>:8080`. En Pop!_OS con `ufw` activo: `sudo ufw allow 8080/tcp`.

8.2 Elementos de la pantalla

Elemento	Contenido
Encabezado	Estado general: **Sano** (3 nodos con réplica síncrona) / **Degradado** (2 de 3) / **Sin primario** (escrituras en pausa)
Cambios por minuto	Ventana móvil de 60 s: ✓ si llega a 90, ⏸ sin carga, ⏳ midiendo (la carga empezó hace menos de 60 s)
Primario	Nodo, IP y timeline actuales
Integridad	Suma de saldos, secuencias faltantes y total de movimientos
Último failover	Duración de la pausa de escrituras medida desde el cliente, hora y nuevo primario
Tarjetas de nodo	Rol, estado, timeline, lag, movimientos y suma **de ese nodo**, salud de etcd, a qué nodo envía HAProxy las escrituras y el botón ★ **Hacer primario**
Replicación	Diagrama: flecha continua = síncrona, discontinua = asíncrona, ✕ = caído
Gráfica	Cambios por minuto de los últimos 10 min en ventanas de 10 s, línea del mínimo (90) y marcas ⚡ de failover (detalle en el tooltip)
Eventos	Caídas, regresos, cambios de rol y de timeline, failovers con su duración, switchovers solicitados
Generadores	Filas, último `seq`, faltantes y si está activo
Últimos movimientos	Las 10 transferencias más recientes y el nodo donde se escribió cada una

8.3 Cómo funciona

- Cada segundo consulta en los 3 nodos la API de Patroni (`/patroni`, `/cluster`), etcd (`/health`), las estadísticas de HAProxy y PostgreSQL directamente.
- Escribe y lee por el mismo camino que la aplicación, lo que le permite medir la pausa real de escrituras.
- El historial de eventos y las marcas de failover viven en memoria: se pierden si se reinicia el panel. La gráfica se reconstruye desde la base.
- ⚠ **Sin autenticación:** cualquiera en la red Tailscale puede usar el botón. En paneles compartidos o proyectados, use `--solo-lectura`.
- Si el panel corre en un nodo que queda aislado por una partición, mostrará "Sin primario" aunque el resto del clúster sí tenga. Hay que interpretar el panel del lado con mayoría.

9. Cambio de primario

9.1 Desde el panel (recomendado)

1. En la tarjeta del nodo deseado pulse ★ **Hacer primario**.
2. Pulse **Confirmar** antes de 8 s.
3. La barra superior muestra los pasos y el resultado, por ejemplo: "✓ n2 es el primario (1 cambio, 10,1 s)".

Comportamiento:

Caso	Lo que hace el panel
El destino es la réplica síncrona	1 switchover directo: unos 3 s sin escrituras y 10–16 s hasta quedar sano
El destino es la réplica asíncrona	2 switchovers encadenados (primero a la síncrona); unos 22–26 s en total
Hay un nodo caído	Solo permite elegir la réplica síncrona
Ya hay un cambio en curso o el nodo ya es primario	Rechaza la solicitud con un mensaje

9.2 Desde la terminal

```
./scripts/pctl.sh switchover --force      # el Sync Standby pasa a primario
./scripts/pctl.sh list                    # repetir hasta que el nodo deseado sea Leader
```

> Con `synchronous_mode` Patroni rechaza `--candidate` hacia un nodo que no sea la réplica síncrona (`412 candidate name does not match with sync_standby`). Es una protección: así nunca se promueve un nodo al que le falten datos confirmados.

10. Procedimientos ante fallas

10.1 Un nodo se apagó o perdió la red

No hacer nada. El clúster lo resuelve solo:

Si el nodo caído era…	Lo que pasa
Réplica	Sin interrupción. Si era la síncrona, Patroni nombra síncrona a la otra
Primario (apagado brusco)	La réplica síncrona asume en unos 26 s
Primario (detención limpia)	La réplica síncrona asume en unos 3–5 s
Primario (sin red)	Se degrada solo a los ~11 s y el otro lado promueve a los ~22 s

Al volver a encender el equipo, Docker levanta los contenedores (con `restart: unless-stopped` y Docker configurado para arrancar al iniciar sesión) y el nodo se reincorpora:

- si era réplica, recupera el WAL atrasado;
- si era primario, Patroni ejecuta `pg_rewind` y lo arranca como réplica.

Si los contenedores no arrancaron solos, ejecute `./scripts/iniciar.sh`.

10.2 No hay primario ("Sin primario" por más de 1 minuto)

1. Revise el quórum de etcd: `bash docker exec etcd etcdctl --endpoints=http://$N1_IP:2379,http://$N2_IP:2379,http://$N3_IP:2379 endpoint health`
2. **Si hay 2 o 3 sanos,** espere 30 s: Patroni elige primario solo. Si no lo hace, revise `docker compose logs patroni` en la réplica síncrona.
3. **Si solo hay 1 sano,** hay dos nodos caídos y el nodo restante está en solo lectura **por diseño**. Encienda otro nodo y el clúster se recupera solo.

10.3 Un nodo queda en `start failed` o no vuelve a `streaming`

Causas posibles: `pg_rewind` falló, faltan segmentos de WAL (desconexión de muchas horas) o hay datos dañados. Para reconstruirlo desde el primario:

```
./scripts/pctl.sh reinit hadb <nodo> --force
```

Borra los datos de ese nodo y los copia de nuevo con `pg_basebackup`. Tarda segundos con esta base de datos.

10.4 Contraseñas distintas entre nodos

Síntoma: `password authentication failed for user "replicator"` en los logs de la réplica.

1. Corrija el `.env` del nodo con las contraseñas correctas.
2. `docker compose up -d` (recrea Patroni con las variables nuevas).
3. `./scripts/pctl.sh reinit hadb <nodo> --force`.

10.5 etcd: `cluster ID mismatch` o el nodo no se une a etcd

Normalmente lo resuelve `./scripts/iniciar.sh` (§11.2). Para hacerlo a mano, desde un nodo sano:

```
docker exec etcd etcdctl member list                                  # anotar el ID del nodo afectado
docker exec etcd etcdctl member remove <ID>
docker exec etcd etcdctl member add <nodo> --peer-urls=http://<IP>:2380
```

Luego, en el nodo afectado: `docker compose down`, `docker volume rm hadb_etcd-data` y `ETCD_ESTADO_INICIAL=existing` en el `.env`, y por último `docker compose up -d`.

10.6 Disco lleno

PostgreSQL se detiene en ese nodo; si era el primario, hay failover. Libere espacio (`docker system prune` borra imágenes y contenedores que no se usan) y, si el nodo no vuelve solo, aplique §10.3.

11. Reinstalar o reemplazar un equipo

11.1 Conservar la IP de Tailscale

Una reinstalación de Tailscale asigna una IP nueva. Para conservar la anterior y no modificar los otros dos equipos:

1. https://login.tailscale.com/admin/machines → **borrar la máquina vieja**.
2. En la máquina nueva: ⋯ → **Edit machine IPv4** → escribir la IP anterior.

11.2 Reinstalación

```
git clone https://github.com/NohelEstradaP/ha_db_ip.git ~/hadb && cd ~/hadb
./instalar.sh        # mismo nodo, mismas 3 IP, "n" a ¿PRIMER equipo?, pegar las 4 contraseñas
                     # al final: S a "¿Levantar ahora este nodo?"
```

Cuando el clúster ya existe y este nodo no tiene datos de etcd, `iniciar.sh`:

1. Detecta los etcd sanos de los otros nodos.
2. Retira el miembro viejo de este nodo (`etcdctl member remove`) y lo vuelve a agregar (`member add`).
3. Guarda `ETCD_ESTADO_INICIAL=existing` en el `.env` y levanta los contenedores.
4. Patroni copia la base desde el primario (`pg_basebackup`) y el script termina cuando el nodo está en `streaming`.

Probado borrando todos los datos de n1 (`docker compose down -v`): el nodo quedó sincronizado en 24 s sin intervención.

11.3 ⚠ Borrar los datos de un nodo a propósito

```
docker compose down -v         # borra PostgreSQL y etcd de ESTE nodo
./scripts/iniciar.sh           # lo reincorpora y copia los datos de nuevo
```

Nunca en el **primario**: primero muévalo a otro nodo (§9).

12. Pruebas de alta disponibilidad

12.1 Procedimiento general

1. Clúster sano (`./scripts/estado.sh`) y carga activa en al menos 2 equipos durante 2 minutos.
2. Panel abierto en un equipo que **no** sea el que se va a afectar.
3. Anote la hora y quién es el primario.
4. Provoque la falla.
5. Observe en el panel el estado "Sin primario", el nuevo primario y la duración del failover.
6. Deje el nodo caído al menos 2 minutos y confirme ≥ 90 cambios por minuto y suma correcta.
7. Restaure el nodo y espere a que esté en `streaming`.
8. Detenga la carga (`./scripts/carga.sh detener` en cada equipo) y ejecute `python app/verificar.py --esperar`.
9. Guarde las evidencias: `bash ./scripts/pctl.sh history > evidencias/pX_historial.txt docker compose logs --since 30m patroni > evidencias/pX_patroni_$(hostname).log python app/verificar.py --esperar > evidencias/pX_verificar.txt`

12.2 Catálogo de pruebas

Prueba	Cómo provocarla	Cómo restaurar	Resultado esperado (medido)
P1	Apagar una réplica desde el menú del sistema	Encenderla	Sin interrupción
P2	`docker kill etcd patroni haproxy` en una réplica	`./scripts/iniciar.sh`	Sin interrupción; streaming en 4 s
P3	Apagar el **primario** con el botón de encendido, o `docker kill etcd patroni haproxy`	Encender / `iniciar.sh`	Nuevo primario en ~26 s; `pg_rewind` al volver
P4	Quitar la red al **primario**: `sudo tailscale down` (Linux) o desde la app (macOS)	`tailscale up`	Se degrada a los ~11 s; nuevo primario a los ~22 s
P5	Quitar la red a una réplica	Reconectar	Sin interrupción
P6	`sudo reboot` en un nodo	Esperar	Vuelve solo como réplica
P7	Dejar una réplica apagada 10 min o más con carga	Encender	Se pone al día por WAL (600 tx en 5 s)
P8	Botón **Hacer primario** o `pctl switchover --force`	—	~3 s sin escrituras
P9	`docker compose stop patroni` en el primario	`docker compose start patroni`	Nuevo primario en ~3 s

13. Laboratorio local

Simula los 3 nodos en un solo equipo con la misma imagen y configuración (red Docker `172.30.0.0/24`), para ensayar sin las 3 máquinas.

```
./laboratorio/lab.sh iniciar          # crea los 3 nodos, la BD y el esquema
./laboratorio/lab.sh carga            # 3 generadores
./laboratorio/lab.sh panel            # http://localhost:18080
./laboratorio/lab.sh apagar n1        # apagado brusco · detener n1 (limpio) · aislar n1 (sin red)
./laboratorio/lab.sh encender n1
./laboratorio/lab.sh primario | estado | logs n1
./laboratorio/lab.sh parar-carga && ./laboratorio/lab.sh verificar
./laboratorio/lab.sh destruir         # borra el laboratorio
```

Paneles de HAProxy del laboratorio: `localhost:17000`, `:27000` y `:37000`. No interfiere con el clúster real (usa otros puertos), pero consume memoria.

14. Mantenimiento

14.1 Actualizar el código en los nodos

```
git pull
```

Si cambió…	Aplicar
`app/`, `scripts/`	Nada más; para el panel o el generador en segundo plano: `--detener` / `detener` y volver a iniciar
`haproxy/haproxy.cfg`	`docker compose restart haproxy` (sin efecto en la BD)
`patroni/Dockerfile` o `patroni/patroni.yml`	Nodo por nodo, **réplicas primero**: `docker compose build patroni && docker compose up -d`; al final mover el primario (§9) y actualizar el último
Parámetros de `bootstrap.dcs` (ttl, synchronous_mode, parámetros de PostgreSQL)	Solo se aplican al crear el clúster. Para cambiarlos en caliente: `./scripts/pctl.sh edit-config` (vale para los 3 nodos)

14.2 Respaldo lógico

```
docker exec patroni pg_dump -U postgres -Fc hadb > respaldo_hadb_$(date +%F).dump    # en cualquier nodo
# Restaurar (en el primario, ⚠ sobrescribe):
docker exec -i patroni pg_restore -U postgres -d hadb --clean < respaldo_hadb_AAAA-MM-DD.dump
```

La replicación protege contra la caída de equipos, **no** contra errores lógicos: un `DELETE` accidental se replica a los 3 nodos.

14.3 Limpiar datos de prueba

Con el clúster sano, desde cualquier equipo:

```
source scripts/env_app.sh
python -c "import os, psycopg; c = psycopg.connect(os.environ['HA_DSN'], autocommit=True);
c.execute('TRUNCATE movimientos'); c.execute('UPDATE cuentas SET saldo = 10000')"
```

14.4 Archivos de registro

Archivo	Contenido
`docker compose logs patroni`	Decisiones de Patroni (líder, promoción, `pg_rewind`) y log de PostgreSQL
`docker compose logs etcd`	Elecciones y miembros de etcd
`evidencias/generador_<nodo>.log`	Salida del generador (reintentos, primario)
`evidencias/generador_<nombre>-<fecha>.csv`	Una fila por transacción: seq, resultado, primario, latencia, intentos
`evidencias/panel.log`	Salida del panel
`evidencias/monitor_<fecha>.csv`	Muestras del monitor de terminal

14.5 Antes de una demostración

- ☐ `git pull` en los 3 equipos.
- ☐ Equipos cargados; reposo desactivado; en las Mac, `caffeinate -dimsu` y tapa abierta.
- ☐ Tailscale conectado en los 3 (`tailscale ping` entre ellos).
- ☐ Actualizaciones automáticas pausadas (Docker Desktop, OrbStack, `apt`).
- ☐ `./scripts/iniciar.sh` en los 3; estado con 1 Leader, 1 Sync Standby y 1 Replica.
- ☐ Generador en cada equipo (`./scripts/carga.sh iniciar`).
- ☐ Panel en dos equipos (o uno más el celular), el compartido con `--solo-lectura`.
- ☐ Primario en el nodo deseado (botón del panel).

15. Referencia de comandos

15.1 Scripts del proyecto

Comando	Descripción
`./instalar.sh [--lan] [--solo-env] [--manual] [-y]`	Prepara el equipo como nodo
`./scripts/iniciar.sh`	Levanta el nodo, lo reincorpora si hace falta y espera a que esté sincronizado
`./scripts/estado.sh`	Patroni, etcd y primario según la API REST
`./scripts/crear_bd.sh`	Usuario, base y esquema (idempotente)
`./scripts/probar_puertos.sh`	Conectividad a los 6 puertos de los 3 nodos
`./scripts/carga.sh iniciar [--tps N] | detener | estado`	Generador en segundo plano
`./scripts/panel.sh [--fondo | --detener] [--solo-lectura] [--puerto P]`	Panel web
`./scripts/pctl.sh <args>`	`patronictl` dentro del contenedor
`source scripts/env_app.sh`	Prepara la terminal (venv, variables, `HA_DSN`) para `app/*.py`
`./scripts/secretos.sh guardar | leer`	Opcional: cifra la configuración compartida en `cluster.env.enc`
`python app/generador.py [--tps N] [--nombre X]`	Generador en primer plano
`python app/verificar.py [--esperar]`	Verificación de integridad
`python app/monitor.py`	Monitor de terminal

15.2 patronictl (vía `./scripts/pctl.sh`)

Comando	Uso
`list`	Miembros, rol, estado, timeline, lag
`history`	Historial de timelines (cada promoción)
`topology`	Árbol de replicación
`switchover --force`	Cambio planificado al Sync Standby
`failover --candidate <nodo> --force`	Forzar failover (emergencias)
`reinit hadb <nodo> --force`	Reconstruir una réplica desde el primario
`restart hadb <nodo>`	Reiniciar PostgreSQL en un nodo
`show-config` / `edit-config`	Ver o editar la configuración dinámica (DCS)
`pause` / `resume`	Suspender o reanudar el failover automático (mantenimiento)

15.3 etcd, PostgreSQL, Docker y Tailscale

```
# etcd
docker exec etcd etcdctl member list -w table
docker exec etcd etcdctl --endpoints=http://$N1_IP:2379,http://$N2_IP:2379,http://$N3_IP:2379 endpoint
status -w table
```

```
# PostgreSQL (en el primario)
docker exec -it patroni psql -U postgres -c "SELECT application_name, client_addr, state, sync_state,
replay_lag FROM pg_stat_replication;"
# PostgreSQL (en una réplica)
docker exec -it patroni psql -U postgres -c "SELECT pg_is_in_recovery(), pg_last_wal_replay_lsn();"

# API REST de Patroni
curl -s http://$N1_IP:8008/patroni | jq
curl -s http://$N1_IP:8008/cluster | jq
curl -s -o /dev/null -w "%{http_code}\n" http://$N1_IP:8008/primary     # 200 = primario

# Docker
docker compose ps
docker compose logs -f --tail 100 patroni
docker compose restart haproxy

# Tailscale (en macOS: /Applications/Tailscale.app/Contents/MacOS/Tailscale)
tailscale status
tailscale ip -4
tailscale ping <IP>
tailscale netcheck
```

Para usar `$N1_IP` en la terminal: `set -a; source .env; set +a`.

16. Solución de problemas

Síntoma	Causa probable	Solución
macOS: puertos 5000/7000 ocupados (`ControlCe`)	Receptor AirPlay	Desactivarlo (§5.1)
`iniciar.sh` espera el quórum de etcd para siempre	Los otros nodos no están arriba o no se alcanzan	`tailscale ping`, `./scripts/probar_puertos.sh`, revisar las IP del `.env`
`probar_puertos.sh`: todo FALLA	El clúster no está levantado, o Tailscale está desconectado en este equipo	`./scripts/iniciar.sh`; `tailscale status`
`password authentication failed`	Contraseñas distintas entre `.env`	§10.4
Nodo en `start failed`	`pg_rewind` falló o falta WAL	§10.3
Nodo en `stopped` varios minutos, con `running pg_rewind … in progress` en el log	`pg_rewind` colgado por un corte de red (no tiene tiempo límite)	`./scripts/pctl.sh reinit hadb <nodo> --force` (§10.3)
`cluster ID mismatch` en etcd	Volumen de etcd borrado o nodo reinstalado	`./scripts/iniciar.sh` (automático) o §10.5
`412 candidate name does not match with sync_standby`	Switchover hacia un nodo que no es la síncrona	Panel o `switchover --force` (§9)
Panel: "Sin primario" pero el clúster funciona	El panel corre en el nodo aislado	Ver el panel en un nodo del lado con mayoría
Panel: "⏸ sin carga"	No hay generadores activos	`./scripts/carga.sh iniciar`
Generador no se detiene con Ctrl+C	Lanzado en segundo plano (ignora SIGINT)	`./scripts/carga.sh detener` o `pkill -TERM -f app/generador.py`
Build de la imagen: `403 Forbidden` en Debian	Proxy de la red	Otra red y volver a correr `./instalar.sh`
`pip: Read timed out`	Red lenta	Volver a correr `./instalar.sh` (ya usa timeout de 120 s)
Docker no arranca tras reiniciar (Pop!_OS)	Servicio deshabilitado	`sudo systemctl enable --now docker`
Docker no arranca tras reiniciar (Mac)	Sin inicio automático	Docker Desktop u OrbStack: "Start at login"
La Mac se durmió	Reposo	Despertarla: vuelve sola; `caffeinate -dimsu` durante la demo
`docker` pide `sudo` en Pop!_OS	Usuario recién agregado al grupo	Cerrar sesión y volver a entrar
Las IP cambiaron (modo `--lan`)	etcd guarda las IP de sus pares	Plan de implementación §6.4 (reconstruir etcd)

17. Glosario

Término	Definición
Primario (líder)	Nodo que acepta escrituras
Réplica / standby	Nodo que recibe y aplica los cambios del primario; solo lectura
Réplica síncrona	Réplica cuya confirmación espera cada `COMMIT`; la única que puede ser promovida
WAL	*Write-Ahead Log*: registro secuencial de todos los cambios
LSN	*Log Sequence Number*: posición dentro del WAL
Timeline	Rama de la historia del WAL; aumenta en cada promoción
Failover	Cambio automático de primario ante una falla
Switchover	Cambio planificado de primario
`pg_rewind`	Herramienta que alinea un primario viejo con el nuevo copiando solo lo distinto
`pg_basebackup`	Copia completa de la base para crear una réplica
Quórum	Mayoría de miembros necesaria para decidir (2 de 3)
Split-brain	Dos primarios aceptando escrituras al mismo tiempo; esta solución lo impide
DCS	*Distributed Configuration Store*: aquí, etcd
TTL	Tiempo de vida de la llave de líder en etcd (20 s)
RPO / RTO	Datos que se pueden perder / tiempo hasta recuperar el servicio
Idempotencia	Propiedad de una operación que, repetida, produce el mismo resultado que una vez
Tailscale	VPN en malla basada en WireGuard; IP fija `100.x.y.z` por equipo